Fakturama Image-to-Cash Automation

Proposed design | Kareem Abed | 1 October 2026

1. Scope and success criteria

Turn one supplied order image into one saved Order and one linked Invoice through Fakturama's UI. Success requires persisted-record verification, not just successful clicks. The Order must remain open and the Invoice must carry the source payment method and, only for PAID, its payment date and full value. Never create a Delivery, Correction, or Dunning document.

The assignment timeboxes design to 90 minutes and implementation to five hours and permits incomplete implementation. Prioritize a verified vertical slice, then missing-master branches, multi-line orders, and recovery tests. Report actual implementation and evidence separately from this proposed design. Target Windows with Python 3.11 and an inspected Fakturama version; neither accessibility coverage nor screenshots establish that a live integration works.

2. Architecture and image extraction

Use four components: an image extractor, a deterministic validation layer, an Order-first workflow, and a desktop adapter. Keep business rules independent of the desktop so financial calculations and branching can be tested without an installed app. A simulator tests orchestration but cannot prove Fakturama compatibility.

Accept only bounded, local PNG/JPEG files. Check decoded dimensions, reject multi-frame files, apply EXIF orientation, and decode/re-encode to strip metadata. Local OCR provides source text; an optional vision model produces schema-constrained data when OCR layout is insufficient. Keep remote processing explicitly opt-in. Do not upload application screenshots or accounting data as part of extraction.

Extract order date/reference, company/contact/alias, billing and delivery addresses, payment method/status/date, currency, every line's SKU/description/quantity/unit net/VAT/discount/source total, and aggregate totals. Preserve source row order and repeated SKUs. Missing information and uncertain readings become review issues; do not infer dates, countries, currency, addresses, or payment state. Record field evidence where the extractor supports it. A model's self-reported confidence is not proof of correctness.

Treat all image/OCR/model content as data. The extractor receives no desktop actions, shell tools, or arbitrary URLs. Parse output into an allowlisted schema with unknown fields rejected. Reject oversized strings, control characters, non-finite amounts, negative values, impossible dates, invalid percentage ranges, and unsupported payment methods. Human review is required for unclear line-total meaning or unresolved fields before any desktop mutation.

Use decimal arithmetic and an explicit rounding policy. Product gross price is unit net multiplied by one plus VAT/100, rounded to cents; it never includes line discount. Line net is quantity multiplied by unit net multiplied by one minus discount/100. Reconcile line net, VAT, total net, and gross with the image before opening the Order, then with the UI. Verify Fakturama's VAT aggregation and rounding experimentally. Initially reject nonzero order-level discounts/shipping and unsupported currencies until their semantics are implemented and tested; never silently erase them. A differing billing/delivery address requires verified separate address roles.

3. Desktop grounding and continuous workflow

Start by inspecting the actual UI Automation tree on Windows using pywinauto and Microsoft's Inspect tooling. Bind to an explicitly selected process, not the first matching desktop window. Scope locators to the current editor or modal; prefer observed automation IDs, accessible names, types, and parent relationships. Require exactly one visible, enabled match. Use UIA value/invoke/select/toggle patterns instead of keystrokes wherever possible. This avoids source text becoming hotkeys and avoids hardcoded screen coordinates.

If Fakturama's controls are not fully accessible, add OCR grounding inside the current application window: locate a label/row in a fresh screenshot, derive the control rectangle from that observation, and verify focus and readback. Never reuse absolute coordinates or select a guessed first row. Icon-only controls require inspected identity or an unambiguous fresh visual anchor, particularly the existing-contact/product selectors versus the green creation icons. Stop if a control cannot be grounded reliably.

Wait for the expected editor/modal and stable search results with a timeout. Require the search field to reflect the exact query and the result set to be stable after search completion. Distinguish an empty result from a loading/failed search. Retry observation, never an uncertain save or creation. Verify the original Order tab remains present while another editor is open.

The workflow opens New Order first; retains proposed number, sets extracted date/reference, Net and With VAT. From its address selector, match Company, First Name, Name, ZIP, and City exactly after harmless Unicode/whitespace normalization. Do not use fuzzy matching or normalize away identifiers. Duplicate exact rows or conflicting identity require review. After selection, compare complete billing and delivery addresses to the image.

3. Desktop workflow (continued)

If absent, cancel the selector and create a Debtor while retaining the Order. Preserve proposed customer ID; fill only supplied optional fields, address roles, alias, zero discount, and Net mode. Resolve payment from the Debtor dropdown, then the terms list if unavailable. Map Bank Transfer to Credit transfer, Credit Card to Credit card, and SEPA Direct Debit to SEPA direct debit. New terms have exact name/description, blank account/texts, zero cash discount/days/net days, and are not made standard. Save once, return to the Order, and reselect the saved Debtor.

For every line, use the Order product selector and exact SKU. If absent, resolve VAT first: exact name VAT followed by percentage plus %, matching value and S (Standard rate). Stop on conflicting settings or duplicate exact records. Create only missing VAT, preserve Standard VAT, then create the Product with source SKU/description, calculated gross price, zero net cost and stock, and remaining optional fields blank/unchanged. Save once and reselect from the same Order. Set and read back quantity, unit net, VAT, line discount, and line net; do not overwrite an existing Product master to accommodate a transaction price.

Check addresses, all lines, overall discount, shipping, and totals, then save Order once. Verify one Documents row by generated number, date, reference, open state, and total. Reopen that saved Order and use its follow-up Invoice action, never the top-level Invoice toolbar. Preserve proposed Invoice number/date/service date. Verify copied Order date, reference, addresses, modes, lines, totals, and relationship. Set exact payment method; for PAID, set paid/date/full total; otherwise leave paid clear without adding payment date/value. Save once. Verify persisted Invoice details and Documents rows for Invoice and still-open source Order.

4. Recovery, security, and validation

A local SQLite journal records a hashed source/job identity, stage, intended action before mutation, and observed document IDs. Acquire a single-writer workspace lease before desktop changes. Block duplicate jobs, altered input with the same customer/reference, and new runs after an interrupted transaction. A crash between save and verification is an uncertain result requiring manual reconciliation. Do not claim exactly-once guarantees or implement blind retries/rollback: saved records may exist, and automatic deletion of accounting data is inappropriate.

Keep credentials in environment variables, not code, prompts, logs, or screenshots. Use fixed provider endpoints, bounded responses/timeouts, no shell execution of source content, and no database writes to Fakturama. Run with the user's normal desktop permissions. Store evidence locally with restricted access; exclude journals, images, UI dumps, and keys from Git. Log stage/error codes and opaque IDs instead of names/addresses. Default screenshots off; opt-in evidence can contain personal data and must be reviewed before submission. Document dependency versions and audit findings without claiming zero vulnerabilities.

Unit tests cover Decimal rounding, multi-rate/multi-line totals, invalid extraction, conflicting matches, all payment mappings, and creation branch rules. Simulator tests verify Order-first sequencing, open-tab continuity, exact selector reuse, linked Invoice creation, both payment states, saved readback, duplicate prevention, and failure injection before/after saves. Windows acceptance tests use an isolated Fakturama workspace with synthetic data: existing/missing masters, separate addresses, duplicate rows, changed window size/DPI, loading delays, modal interruption, crash recovery, and rounding differences. Capture actual annotated screenshots or a short recording with Order and linked Invoice numbers; never present simulated results as desktop evidence.

Sources: assignment supplied by TJM Labs; Fakturama downloads (https://www.fakturama.info/downloads/); pywinauto getting-started and inspection documentation (https://pywinauto.readthedocs.io/en/latest/getting_started.html). UI locators and accounting behavior remain subject to installed-version validation.